

AnswerDoctor

Reasoning-level diagnosis and batch grading for handwritten answer scripts, with collusion detection built in.

Track(s)	AI/ML & Open Innovation
Domain	LMS infrastructure & script diagnostics
Stage	Review 0 screening submission
Team	trpSurgewave
Members	Mangalapalli Sohum Seshu Krish (26BCE0616, Team Lead), Rayed Rabbanee (26BCE0606), Pratyush Jha (26BCE0604), Turany Pandey (26BCE0646)

1. The problem, as we actually ran into it

When you get a CAT or FAT script back at VIT, you usually get a number and maybe a line struck through in red pen. What you almost never get is the reason — whether the concept was right and the execution slipped somewhere, or the whole approach was off from the first line. That's not really the grader's fault either. Someone marking two hundred scripts in one sitting doesn't have time to write out why each mark was lost; they're moving script to script against a deadline, not question to question across the batch.

Here's the situation that got us thinking about this: a second-year mechanical engineering student loses four marks on a thermodynamics question in a CAT for using the wrong reference state before applying the first law. The answer sheet says “-4, incorrect,” and nothing else. In the semester-end exam, they make the same mistake on a similar question, because the actual error was never pointed out — only the penalty was. Meanwhile, the instructor grading all 240 scripts in that batch has no way of noticing that a third of the class made exactly the same wrong assumption, because grading happens script by script, not concept by concept.

That's the gap we're actually trying to close: not “grade faster,” but let a student see where their reasoning broke and get a shot at fixing it, and let the instructor see it as a class-wide pattern instead of finding out sixty scripts later.

2. Where the tools people already use fall short

Auto-graders like Gradescope are built around a single teacher grading a single class — there's no institutional layer, no batch ingestion mapped to student IDs, and nothing that syncs back into a gradebook across a department. Plagiarism checkers like Turnitin compare plain text, so they're blind to two handwritten derivations that arrived at the same wrong step through the same wrong logic — that's a semantic match, not a text match, and it's exactly the kind of thing a text-similarity tool will never catch. And even the AI tutoring chatbots that do flag a mistake don't let a student go back and drill that one specific step; the gap just resurfaces on the next test, because there's no rubric underneath the conversation to ground it.

3. What we're building

AnswerDoctor is a role-based platform (built on PostgreSQL/Supabase) with two dashboards — one for teachers, one for students — sitting on top of a multi-agent pipeline that looks at how a student reasoned, not just what they wrote at the end.

- Teacher side: batch script upload, a rubric builder, class-wide analytics, and a malpractice radar.
- Student side: joining a classroom, uploading individual scripts, seeing a step-by-step diagnosis, and retrying the specific step that went wrong.

Scripts come in through a batch OCR pipeline that auto-crops and cleans up scanned pages, then keeps diagrams and derivation text separate — a circuit sketch or a free-body diagram is preserved as an image alongside the diagnosis, not force-transcribed into text and mis-scored as a missing step.

The agent pipeline (LangGraph)

- Rubric Decomposer — breaks the marking scheme into atomic units: concept, formula, intermediate step, units, final answer.
- Semantic Alignment Agent — checks the student's text against each rubric unit using sentence-transformer embeddings (similarity threshold $\gamma \geq 0.60$).
- Malpractice & Collusion Radar — cross-clusters embeddings across the whole cohort to catch scripts with suspiciously identical phrasing or shared errors.

On top of a semester's worth of scripts, we also cluster recurring error types (scikit-learn) so a teacher can see which misconceptions keep coming back across tests, not just within one.

4. The core idea: the Reasoning Map

Every answer gets represented as a structured map — concept, approach, steps, transformation, result — instead of just a final mark. That one structure is what lets us build two different views out of the same data.

For the student: instead of “Step 3 is wrong,” the tool says something like “your error starts here — you applied the formula before establishing the required condition,” then gives a short practice question on exactly that.

For the teacher: a view of how the whole class approached a question, and which misconceptions are common enough to be worth addressing in the next lecture rather than the next exam.

5. How this is different from what already exists

Tool category	Where it stops short
Auto-graders (Gradescope-style)	Score the final answer, not the derivation. No step-level reason for the mark, no view across a batch.
Plagiarism checkers (Turnitin-style)	Only compare plain text. Two handwritten scripts that made the same wrong reasoning leap look completely unrelated to them.
Generic LLM tutoring bots	Not grounded in the actual rubric. Can explain a concept in general, but can't grade against what the instructor asked for, and has no batch or teacher-side view.

6. Scoring: RAS and CMI

Rubric-Alignment Score (RAS)

$$\text{RAS} = [\sum (\text{Matched_Units} \times \text{Unit_Weight}) / \sum (\text{Total_Units} \times \text{Unit_Weight})]$$

Any unit scoring below $\gamma = 0.60$ similarity gets flagged “Missing / Weak” rather than silently marked wrong.

Cohort Malpractice Index (CMI)

$$\text{CMI}_{ij} = \text{CosSim}(\text{Emb}_i, \text{Emb}_j) \times \text{ErrorPatternMatch}(S_i, S_j)$$

A CMI above 0.88 between a pair of scripts flags them for a teacher to look at — it doesn't accuse anyone on its own.

7. Tech stack

Layer	Technologies	What it's for
-------	--------------	---------------

Agentic AI swarm	LangGraph, Gemini 2.5 Flash / Groq	Rubric decomposition, step-level diagnosis, malpractice auditing
Handwriting vision	Google Vision API, OpenCV, Tesseract	Digitising batch and single handwritten scripts, with spatial bounding-box alignment
Semantic alignment	sentence-transformers (all-MiniLM-L6-v2)	Deterministic vector matching, so grading is explainable and repeatable
Database & auth	PostgreSQL (Supabase), SQLAlchemy, Redis	Role-based access for teachers/students, class cohorts, script history
Backend & analytics	FastAPI, WebSockets, Pandas, scikit-learn	Batch grading workers, live RAS computation, error clustering
Web app	Next.js, React, Tailwind CSS, Recharts	Teacher grading cockpit and student diagnosis/retry view

8. 48-hour MVP — who's building what

Component	Owner	Status	Scope
Dual-role auth & classrooms	Member 2	Core	PostgreSQL-backed teacher/student accounts; students join a classroom via a join code
Batch & individual script OCR	Member 2	Core	FastAPI endpoint for single uploads and teacher batch ZIP submissions
Agentic rubric decomposition	Member 1	Core	LangGraph agent that breaks an answer scheme into atomic, gradeable units
Diagnostic UI	Member 3	Core	Report with matched units in green, missing steps in red, plus the Reasoning-Map view
Step-level retry practice	Member 4	Core	Student retries the one reasoning step that failed, with a targeted follow-up question
Alternative-solution discovery	Member 4	Stretch	Surfaces other valid approaches found across the submitted scripts
Malpractice similarity radar	Members 1 & 2	Stretch	Pairwise embedding comparison flagging near-identical mistakes across a batch

9. Demo flow

- Step 1: teacher logs in, creates a class, uploads a batch of mock scripts plus the answer key.
- Step 2: system OCRs the scripts, aligns them against the decomposed rubric, and flags two scripts as a likely collusion pair.
- Step 3: a student logs in, opens their Reasoning Map with green/red step diagnostics, and sees their own weakness trend.
- Step 4: the student retries the flagged step via a targeted follow-up question; the teacher checks the class-wide weakness heatmap and the alternative solutions the Discovery Agent found.

10. Questions we expect to get asked

“Isn't this just an LLM grading scripts with extra steps?”

Not really — the grading itself isn't an LLM judgment call. The Semantic Alignment Agent scores each step against the rubric with fixed sentence-transformer embeddings and a similarity threshold, so the same input gives the same score every time. The LLM only comes in afterwards, to put the already-identified error into words a student can act on.

“How do you keep the diagnosis from hallucinating?”

The Diagnosis Agent only ever explains the specific rubric unit and step that the alignment agent has already flagged as a mismatch — it's not free-reasoning over the whole script, it's writing up something that's already been located and grounded in the actual student text.

“What if the handwriting is messy, or the answer isn't just text?”

Every extracted step carries an OCR confidence score. Anything low-confidence gets routed to a teacher for manual review instead of being auto-graded, so bad handwriting degrades into a human check rather than a silent wrong mark. Diagrams and multi-line math are detected and kept as image crops next to the diagnosis, and left out of the text-based scoring entirely, so a diagram never gets mis-scored as a missing derivation step.

“Could the malpractice radar flag innocent students?”

The 0.88 threshold is a trigger for a teacher to look at a pair of scripts side by side, not an automatic penalty. It's cutting 240 scripts down to a short review list — the teacher still makes the call.

“Why would a college switch from what it already uses?”

It doesn't have to be a switch — AnswerDoctor is meant to sit next to whatever LMS a college already has and add the reasoning-level diagnosis and cohort view that neither auto-graders nor plagiarism tools currently give it.

“Is student handwriting data actually safe?”

Access is role-based and scoped to a student's own enrolled classrooms. Scripts stay in a scoped PostgreSQL instance as institution-owned data — they aren't sent anywhere outside the pipeline's own agent calls.

11. One-line pitch

AnswerDoctor doesn't just mark whether an answer is right — it shows a student where their reasoning broke, gives them a way to fix it, and shows the teacher where the whole class is making the same mistake.